📄 STUDY BUDDY – FULL PROJECT DOCUMENTATION

🧠 1. PROJECT OVERVIEW

Project Name: Study Buddy
Type: Student–Teacher Course Connection Platform

🎯 Objective:

A platform where:

- Students browse courses and request enrollment
- Teachers review and accept/reject students
- Admin approves teachers and handles complaints

👥 2. USER ROLES

👨‍🎓 Student

- Register/Login
- Browse courses
- Apply for courses
- Save notes
- File complaints against teachers

👨‍🏫 Teacher

- Create profile (pending approval)
- Add courses
- View student requests
- Accept/Reject requests

🛡️ Admin

- Approve/reject teachers
- View complaints
- Resolve complaints

🔁 3. SYSTEM WORKFLOW

🧩 STUDENT FLOW

1. Student registers → becomes `Student`
2. Views available courses
3. Applies for course
4. Request stored in `Enrollment_Request`
5. Teacher reviews request
6. If accepted → student continues learning (outside system)
7. Student can:
    - Save notes
    - File complaints

🧩 TEACHER FLOW

1. Teacher registers → `is_approved = N`
2. Admin approves → `is_approved = Y`
3. Teacher creates courses
4. Receives enrollment requests

5. Accepts/rejects requests

🧩 ADMIN FLOW

1. Reviews teacher profiles
2. Approves teachers
3. Views complaints
4. Clicks “Resolve”
5. Complaint moves to resolved

🧱 4. DATABASE SCHEMA (FINAL)

🔷 Users (Superclass)

user_id (PK)
name
email (UNIQUE)
password
created_at

👨‍🎓 Student

user_id (PK, FK → Users)

👨‍🏫 Teacher

user_id (PK, FK → Users)
is_approved (ENUM: Y/N)

🛡️ Admin

user_id (PK, FK → Users)
role

📘 Course

course_id (PK)
title (UNIQUE)
description
teacher_id (FK → Teacher)
category_id (FK → Category)

🏷️ Category (Lookup Table)

category_id (PK)
name (UNIQUE)
description
age_group
created_at

🔁 Enrollment_Request

request_id (PK)
student_id (FK)
course_id (FK)
status (ENUM: pending, accepted, rejected)
message
request_date

⚠️ Complaint

complaint_id (PK)
student_id (FK)
teacher_id (FK)
description
status (ENUM: pending, resolved)
created_at

resolved_at
resolved_by (FK → Admin)

📝 Notes (Weak Entity)

note_id (partial key)
student_id (FK)
course_id (FK)
content
created_at

PRIMARY KEY (note_id, student_id)

✅ Teacher_Approval (1:1)

approval_id (PK)
teacher_id (UNIQUE FK)
approved_by (FK → Admin)
status (ENUM: approved/rejected)
approved_at

🔗 5. RELATIONSHIPS

- User → Student / Teacher / Admin (Specialization)
- Teacher → Course (1:M)
- Category → Course (1:M)
- Student ↔ Course (M:N via Enrollment_Request)
- Student → Complaint (1:M)
- Teacher ← Complaint (1:M)
- Admin → Complaint (1:M)
- Student → Notes (1:M)
- Course → Notes (1:M)
- Teacher ↔ Teacher_Approval (1:1)

⚙️ 6. CORE FUNCTIONALITIES (TO IMPLEMENT)

🔹 AUTH SYSTEM

- Register (Student / Teacher)
- Login
- Role-based routing

🔹 STUDENT FEATURES

- View courses
- Apply for course
- View application status
- Add notes
- File complaint

🔹 TEACHER FEATURES

- Create course
- View requests
- Accept/Reject student

🔹 ADMIN FEATURES

- Approve teacher
- View complaints
- Resolve complaint

🧪 7. REQUIRED QUERIES IMPLEMENTED

- SELECT (filters)
- GROUP BY (aggregates)
- Subqueries (including correlated)
- JOINs (inner, left, multi-table)
- UPDATE / DELETE
- GRANT / REVOKE

📊 8. DATA REQUIREMENTS

- ≥ 20 rows in major tables
- ≥ 10 rows in Category
- Realistic data (no dummy values)

🧠 9. IMPORTANT DESIGN DECISIONS

✔ Why ENUM instead of CHECK?

MySQL compatibility (CHECK not always enforced)

✔ Why no teacher_id in Enrollment_Request?

Avoid redundancy — teacher derived via Course

✔ Why Notes is weak?

Depends on Student (partial key)

✔ Why one admin?

Simplifies system (central authority)

✔ Why Teacher_Approval table?

To satisfy 1:1 relationship and track approvals

🖥️ 10. FRONTEND REQUIREMENTS

Student UI

- Course list
- Apply button
- Notes section
- Complaint form

Teacher UI

- Course creation form
- Requests dashboard

Admin UI

- Teacher approval panel
- Complaint dashboard
- Resolve button

🔌 11. BACKEND REQUIREMENTS

API Endpoints (Suggested)

Auth

- POST /register
- POST /login

Student

- GET /courses
- POST /apply
- POST /notes
- POST /complaint

Teacher

- POST /course
- GET /requests
- PUT /request/status

Admin

- GET /teachers
- PUT /approve-teacher
- GET /complaints
- PUT /resolve-complaint

🔐 12. SECURITY

- Role-based access
- Input validation

🚀 13. DEVELOPMENT NOTES (IMPORTANT FOR AI TOOLS)

Tell Cursor / Antigravity:

👉 Follow this strictly:

- Use given DB schema exactly
- Maintain FK constraints
- Use meaningful joins (no raw IDs in UI)
- Normalize data (no duplication)
- Implement role-based logic

🎯 FINAL GOAL

A working system where:

- Students connect with teachers
- Teachers manage courses
- Admin controls platform integrity
